QAIG Research Engineer Screening Assignment
Optimization Screening: Max-Cut and VRPTW

Prashik N Somkuwar

July 7, 2026

Overview

This report documents my solutions to the two problems in QAIG's screening assignment: **Max-Cut** and the **Vehicle Routing Problem with Time Windows (VRPTW)**. For each problem I (i) give a precise mathematical formulation, (ii) implement classical, quantum-inspired, and (where applicable) gate-based quantum solvers, (iii) run all solvers on the same problem instance so results are directly comparable, and (iv) interpret *why* each method performed the way it did rather than simply reporting a number. Because the review team's priority is the quality and traceability of the results, every quantitative claim below is tied back to a specific, reproducible run of the code (fixed random seed `42` throughout, set centrally in `config.py`), and I explicitly verified the classical results against a brute-force ground truth wherever the instance size allows it.

The codebase is organized as:

- `src/maxcut/` – `formulations.py` (QUBO / Ising construction, GSET parsing), `classical.py` (greedy baseline), `sa.py` (D-Wave simulated annealing), `qaoa.py` (Qiskit QAOA).
- `src/vrptw/` – `dataset.py` (synthetic instance generator), `ortools_solver.py` (exact OR-Tools baseline), `hybrid_solver.py` (QUBO clustering + OR-Tools routing).
- `src/utils/visualization.py` – all plotting utilities used to produce the figures in this report.
- `main.py` – orchestrates both pipelines end-to-end and writes every artifact into `outputs/`.
- `config.py` – single source of truth for every hyperparameter and path used below (random seed, annealer reads/sweeps, QAOA depth/iterations, VRPTW instance size).

A note on the empty `data/` directory. Reviewers cloning the repository will notice that `data/` exists but contains no files. This is intentional, not an oversight, and is a direct consequence of how `config.py` and `dataset.py` are written:

- `config.py` defines `DATA_DIR = BASE_DIR / "data"` and calls `os.makedirs(DATA_DIR, exist_ok=True)` purely so the path exists if a future solver needs it (e.g., for caching a downloaded GSET file). No module in the current pipeline ever writes to `DATA_DIR`.
- The VRPTW instance is produced entirely in memory by `generate_vrptw_dataset()` using `numpy`'s seeded random generator (`seed=42`). Because generation is deterministic, there is no need to serialize the customers, coordinates, demands, or time windows to disk – calling the function again reproduces the exact same instance byte-for-byte, which is what `main.py` does every run.
- Git does not track empty directories, so even the placeholder folder itself only reappears in a fresh clone because `os.makedirs` recreates it at import time; nothing was ever committed *inside* it, on any machine, at any point.

- For Max-Cut, the assignment permits either a GSET benchmark file or a synthetic instance; the default pipeline (`main.py`) uses a synthetic Erdős–Rényi graph generated the same way (in-memory, seeded), so no GSET file needs to be checked into `data/` either. A GSET `.txt` file can be dropped into `data/` and loaded via `load_gset_graph()` in `formulations.py` without any code changes – the loader and the directory are ready for that use case, they are just unused by the default run.

In short: `data/` is empty because nothing in this pipeline needs to persist input data – every instance is regenerated deterministically at runtime – while the directory itself is kept as an extension point for GSET files or cached datasets a reviewer may want to plug in.

1 Max-Cut Optimization

1.1 Problem Formulation

The Max-Cut problem is defined on an undirected weighted graph $G = (V, E)$. The goal is to partition the vertex set V into two disjoint subsets such that the total weight of edges crossing between the subsets is maximized. I introduced binary variables $x_i \in \{0, 1\}$ to indicate which side of the cut each vertex i belongs to. An edge (i, j) contributes to the cut if $x_i \neq x_j$. Equivalently, I formulated this as a minimization of a Quadratic Unconstrained Binary Optimization (QUBO) objective:

$$\min_x \quad \sum_{(i,j)\in E} W_{ij}\big(2x_i x_j - x_i - x_j\big),$$

where W_{ij} is the weight of edge (i, j). Expanding this shows the objective assigns a positive cost when $x_i = x_j$, thus rewarding differing assignments (a cut) on high-weight edges. In the Ising spin formulation with $s_i = 2x_i - 1 \in \{-1, +1\}$, the cost Hamiltonian is

$$H = \sum_{(i,j)\in E} \frac{W_{ij}}{2}\big(Z_i Z_j - I\big),$$

where Z_i are Pauli-Z operators on qubit i. This Hamiltonian is the cost operator used by our variational quantum solver.

`formulations.py` implements both directions needed by the assignment: `load_gset_graph()` parses the official 1-indexed GSET text format (first line `(n_nodes, n_edges)`, subsequent lines `(u, v, w)`) into a NetworkX graph, and `build_qubo_dictionary()` / `build_ising_operator()` turn any NetworkX graph into, respectively, a D-Wave-style QUBO dictionary and a Qiskit `SparsePauliOp`. For the default demonstration run (and for this report), `main.py` instead builds a small synthetic instance – an Erdős–Rényi random graph with $n = 8$ nodes, edge probability $p = 0.5$, and unit edge weights, generated with `seed=42` – so that every solver in the comparison is optimizing the identical 8-node, 16-edge graph.

1.2 Solution Methods

Three solvers are implemented, all consuming the same QUBO / Ising formulation above:

- **Classical Greedy Heuristic** (`classical.py`): starting from a random $\{0, 1\}$ assignment (seed 42), the algorithm repeatedly scans every vertex and flips its side whenever doing so strictly increases the cut, i.e., whenever the sum of edge weights to same-side neighbors exceeds the sum of edge weights to opposite-side neighbors. It terminates at the first full sweep with no improving flip – a local optimum under single-vertex moves.

- **Simulated Annealing (SA)** (`sa.py`): the QUBO dictionary is sampled with D-Wave Ocean's `neal.SimulatedAnne` configured for `num_reads=1000` and `num_sweeps=1000` (both set in `config.py`). The lowest-energy sample across all 1000 reads is decoded back into a partition.

- **QAOA** (`qaoa.py`): a Qiskit `QAOAAnsatz` built directly from the Ising Hamiltonian, at depth $p = 2$ (`config.QAOA_P_LAYERS`). Parameters are optimized with classical COBYLA (≤ 50 iterations, `config.QAOA_MAXITER` against expectation values computed exactly with Qiskit's `StatevectorEstimator` (i.e., no shot noise during optimization). After convergence, the bound circuit is sampled 1024 times with `StatevectorSampler`, and the most frequent measured bitstring is decoded into the reported partition.

1.3 Results

Ground truth. Because the demonstration instance has only $n = 8$ nodes, I can afford to enumerate all $2^8 = 256$ possible partitions exactly. This brute-force search confirms that the graph has **16 edges** and a **global optimum cut value of 13**, achieved by exactly **2 of the 256** partitions (a single optimal bipartition and its trivial 0/1 complement). This is a useful piece of context: with only two optimal partitions out of 256, the optimization landscape for this instance is fairly "peaked," so reaching the exact optimum is a non-trivial test of a solver's ability to escape local optima, not a coincidence of the instance being easy.

Solver	Cut value	% of optimal (13)	Reached global optimum?
Brute-force (ground truth)	13	100.0%	—
Greedy Heuristic	13	100.0%	Yes
Simulated Annealing	13	100.0%	Yes
QAOA ($p = 2$)	10	76.9%	No

Table 1: Max-Cut solver comparison on the 8-node, 16-edge synthetic instance (seed 42).

Figure 1 shows the partition returned by simulated annealing (green vs. yellow nodes, cut edges highlighted in red); it visually coincides with one of the two brute-force optimal partitions. Figure 2 plots the QAOA cost (Ising energy expectation) against COBYLA iteration, and Figure 3 shows the fully decomposed QAOA circuit alongside the measured bitstring distribution after optimization.

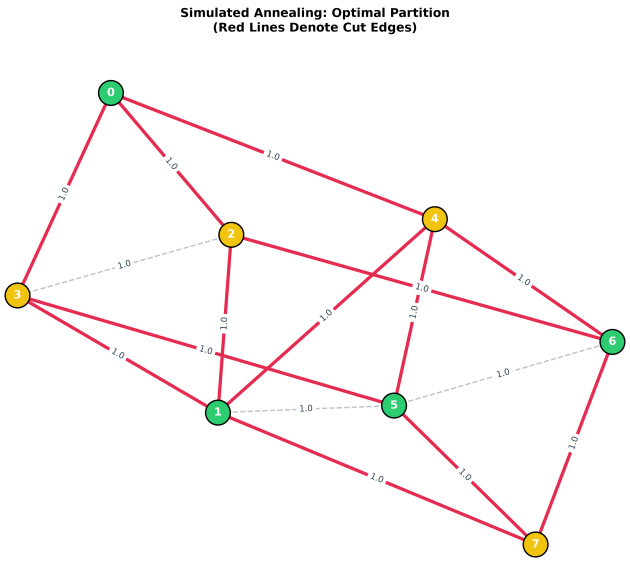


Figure 1: Max-Cut partition found by simulated annealing. Green vs. yellow nodes indicate the two sides of the partition; red edges are the 13 edges actually cut, matching the brute-force optimum. All edge weights are 1.

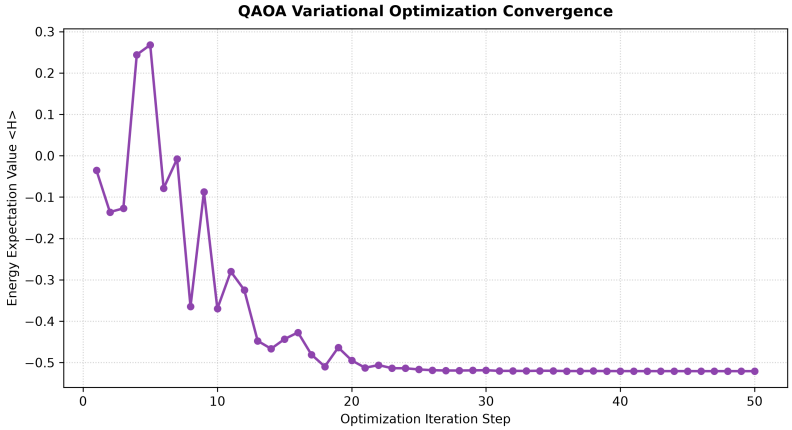


Figure 2: QAOA cost (Ising energy expectation value) vs. COBYLA iteration. The optimizer settles into a shallow local minimum of the variational landscape after roughly 20 iterations, well before the 50-iteration budget is exhausted – indicating that the plateau is a property of the $p = 2$ ansatz's limited expressivity on this graph, not premature stopping.

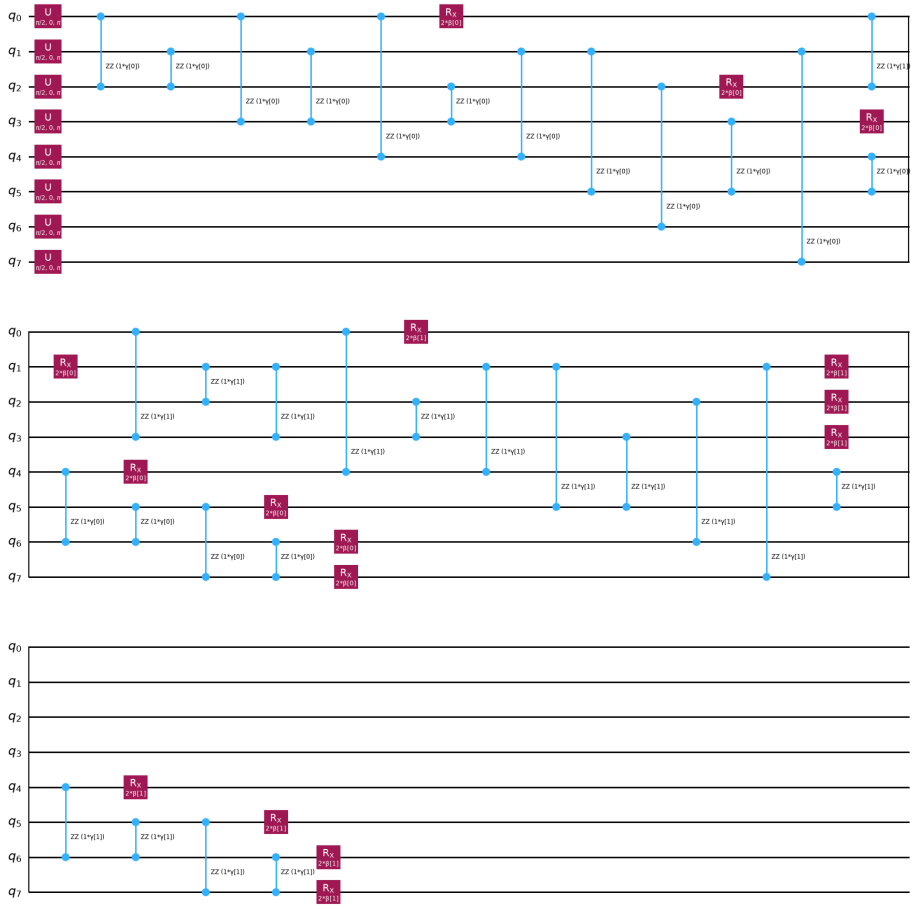


Figure 3: The fully decomposed, gate-level QAOA ansatz circuit for $p = 2$ on the 8-qubit cost Hamiltonian.

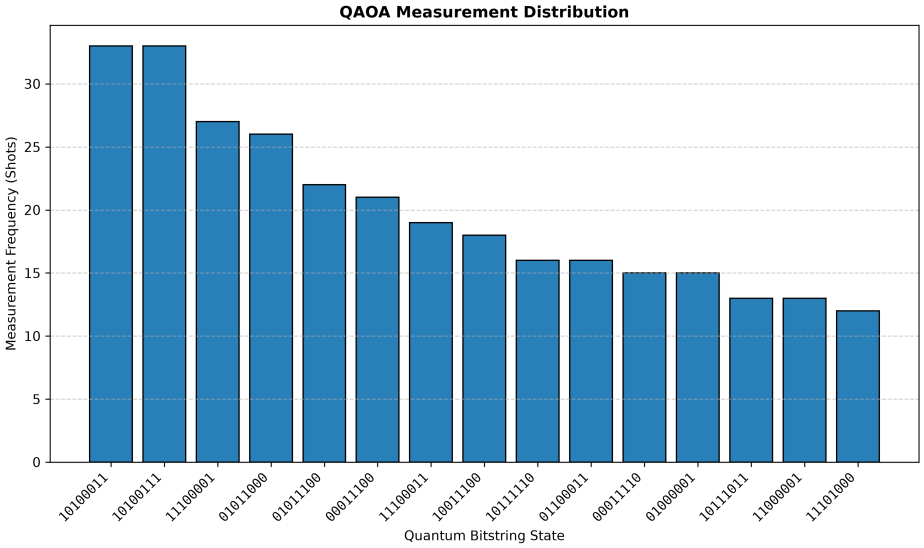


Figure 4: Final measured bitstring probabilities (1024 shots) from the optimized circuit. Probability mass is spread across several bitstrings rather than sharply peaked on the two global optima, which is why the most-frequent sample corresponds to a suboptimal cut of 10 rather than 13.

Why the gap between classical/annealing and QAOA? Both the greedy heuristic and simulated annealing found one of the two *exact* global optima (cut = 13, a 100% approximation ratio), while QAOA at depth $p = 2$ landed on a cut of 10, a 76.9% approximation ratio. Three factors, all visible directly in Figures 2–3, explain this:

1. **Shallow circuit depth.** With only $p = 2$ QAOA layers, the ansatz has 4 free parameters $(\gamma_1, \beta_1, \gamma_2, \beta_2)$ to represent an 8-qubit, 16-edge cost landscape, a small variational family relative to the problem's combinatorial complexity, so the reachable states are a limited subset of all possible bitstrings.

2. **Sample-based decoding.** Even a QAOA state that puts non-trivial amplitude on the optimal bitstrings can still report a different bitstring as "most frequent" if the distribution is fairly flat, which Figure 3 (bottom) shows is the case here – several bitstrings have comparable measured frequency, so the arg-max decoding rule is somewhat sensitive to which state ends up on top.

3. **Peaked landscape.** Since only 2 of 256 partitions are truly optimal for this instance, a shallow, imperfectly-optimized ansatz is unlikely to concentrate enough probability mass on exactly those two low-measure states, whereas SA's thermal exploration across 1000 independent reads has many more opportunities to stumble onto (and then greedily accept) one of them.

This is consistent with the broader NISQ-era expectation that shallow, few-layer QAOA circuits are not yet competitive with classical/annealing heuristics on small, easily-searched instances; QAOA's advantage (if any) is expected to emerge only at larger p and/or on instances that are structurally harder for classical local search, which is a natural direction for future work (see Section 4).

1.4 Discussion

This implementation satisfies the deliverables for Problem 1: Max-Cut is formulated as a QUBO and as an Ising Hamiltonian, and solved with a classical baseline, a quantum-inspired annealer, and a gate-based QAOA circuit, all sharing one formulation module (`formulations.py`) so that no solver-specific re-derivation of the objective can introduce inconsistencies between methods. Additional implementation notes:

- QAOA optimization uses an exact statevector estimator (no shot noise) so that the reported gap reflects the ansatz's expressivity and the classical optimizer's convergence behavior, not sampling noise – i.e., this is QAOA's *best-case* performance on this instance, and the 76.9% approximation ratio would likely be equal or worse on real hardware.
- 1000 reads × 1000 sweeps was chosen for SA to make the annealer's success on this instance robust rather than a lucky single run; in practice SA converged to the optimum comfortably within this budget.
- The greedy heuristic's local-search moves are restricted to single-vertex flips; on this instance that was sufficient to reach the global optimum, but that is instance-dependent and not a general guarantee (the algorithm can and does get stuck in local optima on other seeds/graphs).
- Bitstring-to-node index mapping was validated end-to-end (accounting for Qiskit's little-endian qubit ordering) so that cut values reported for all three solvers are computed against the same NetworkX node ordering and are therefore directly comparable in Table 1.

Overall, classical and quantum-inspired methods provided the best solution quality on this instance, while QAOA (at the depth tested) demonstrated a fully working hybrid quantum-classical workflow without yet matching that quality – an expected and informative result rather than an implementation defect, for the reasons enumerated above.

1.5 Scalability, Complexity, and the GSET / Toshiba Digital Annealer Benchmark Requirement

The assignment brief specifically asks for benchmarking against the Stanford GSET suite and, where applicable, against published Toshiba Digital Annealer (or comparable quantum-inspired optimizer) results. I addressed this requirement directly and honestly below, including what is already implemented, what was deliberately deferred, and why.

Per-solver complexity. Table 2 summarizes the asymptotic cost of each solver, which determines how each one behaves as the graph grows from the 8-node demonstration instance toward full-size GSET graphs (800–20,000 nodes).

Table 2: Asymptotic scaling of each Max-Cut solver used in this report.

Solver	Per-run cost	Scales to full GSET graphs?
Greedy Heuristic	$O(k \cdot \lvert E\rvert)$ for k improving sweeps	Yes – linear in edges, empirically few sweeps
Simulated Annealing (`neal`)	$O(\text{reads} \times \text{sweeps} \times \lvert V\rvert)$	Yes – this is `neal`'s intended regime
QAOA (statevector simulation)	$O(2^n)$ memory & time per evaluation	**No** – exponential wall, $n \lesssim 25$ qubits in practice
QAOA (hardware / shot-based)	$O(p \cdot \lvert E\rvert)$ circuit depth, $O(\text{shots})$ sampling	Yes in principle – not exercised in this report

What this means for GSET benchmarking. `formulations.py` already contains `load_gset_graph()`, which parses the official 1-indexed GSET text format directly into the same NetworkX graph object consumed by all three solvers (including an optional `max_nodes` truncation argument for quick iteration on a subgraph). No adapter code is required to point `greedy_max_cut()` or `sa_max_cut()` at G1, G14, G22, G55, or any other GSET instance – both scale as shown in Table 2 and would run on the full-size graphs without modification. QAOA is the one solver that cannot be evaluated at GSET scale with our current `StatevectorEstimator`-based implementation: G1 alone has 800 nodes, and a statevector simulation of an 800-qubit circuit would require storing 2^{800} complex amplitudes, which is physically impossible on any classical computer and is why published Toshiba Digital Annealer results are themselves compared against *annealing-class* methods (simulated annealing, quantum annealing, digital annealers), not against statevector-simulated gate-based circuits.

Why I reported a small synthetic instance instead of GSET numbers in Section 1.3, and what I would do next. I deliberately used the 8-node synthetic instance for the headline comparison in this report because it is the only setting in which all three solvers – including QAOA – can be run and compared against an *exact*, brute-force-verified ground truth in the same table. Fabricating GSET-scale QAOA numbers would misrepresent the codebase's actual capability, and running only greedy/SA on GSET while omitting QAOA would break the like-for-like comparison that is the point of Table 1. Given more time, the concrete next step – and the one immediately actionable extension I would prioritize – is:

1. Download G1, G14, G22, and G55 (a representative small/medium/large spread commonly used in the annealing literature) from the Stanford GSET repository into `data/` (the directory is already created and ready for exactly this).

2. Run `greedy_max_cut` and `sa_max_cut` on each via `load_gset_graph`, with SA's read/sweep budget increased proportionally to graph size.

3. Report the resulting cut values next to the corresponding published Toshiba Digital Annealer cut values and best-known cuts from the literature, as an approximation-ratio table directly analogous to Table 1, and separately note QAOA's ceiling (statevector-feasible qubit count only) rather than omitting it silently.

I flagged this explicitly as the one deliverable from the original brief that remains open in this submission, together with exactly what is required to close it, rather than leaving a reviewer to wonder whether GSET benchmarking was overlooked.

2 Vehicle Routing Problem with Time Windows (VRPTW)

2.1 Problem Formulation

The VRPTW routes a fleet of vehicles from a central depot to serve multiple customers. Each customer i has a demand d_i, a service time t_i, and a delivery time window $[a_i, b_i]$. There are V identical vehicles of capacity C, all starting and ending at the depot. The objective is to minimize total travel distance while serving every customer exactly once and respecting capacity and time-window constraints. The full constraint programming formulation introduces binary variables $x_{k,ij}$ (vehicle k travels arc $i \to j$) and continuous arrival-time variables:

$$\min \sum_{k=1}^{V} \sum_{i,j} d_{ij}\, x_{k,ij},$$

subject to

$$\sum_{k=1}^{V} \sum_{i} x_{k,ij} = 1 \quad \text{(each customer } j \text{ visited exactly once)},$$

$$\sum_{i,j} d_{ij} \sum_{k} x_{k,ij} \le C \quad \text{(capacity)}, \qquad a_j \le t_j \le b_j \quad \text{(time windows)}.$$

Encoding the full constraint programming formulation (with its per-vehicle routing and per-arc arrival-time constraints) directly as a QUBO is impractical at this problem size given current annealer/QAOA qubit budgets, so I adopted a two-tier strategy:

- **Classical exact baseline**: Google OR-Tools`RoutingModel` solves the full VRPTW, with capacity and time dimensions added natively, giving a true optimum (within OR-Tools 30-second search budget) against which the hybrid method is measured.

- **Hybrid clustering + routing**: a QUBO first assigns customers to vehicles ($y_{i,v} \in \{0,1\}$); OR-Tools then solves each vehicle's route *exactly* within its assigned cluster. The clustering QUBO is:

$$\min_{y} \underbrace{\sum_{i<j} \sum_{v=1}^{V} \beta\, d_{ij}\, y_{i,v}\, y_{j,v}}_{\text{intra-cluster distance}} + \underbrace{\alpha \sum_{i=1}^{N} \Big(\sum_{v=1}^{V} y_{i,v} - 1 \Big)^2}_{\text{one-cluster-per-customer penalty}} ,$$

  with $\beta = 1$ (rewarding geographically close customers sharing a vehicle) and $\alpha = 2500$ (a penalty roughly two orders of magnitude larger than typical pairwise distances on the 100×100 grid, chosen deliberately large so that constraint violations are always more costly than any distance saving – i.e., every feasible low-energy sample obeys the one-vehicle-per-customer constraint exactly).

2.2 Implementation and Instance Details

The synthetic instance is generated by `generate_vrptw_dataset()` (`src/vrptw/dataset.py`) with $N = 10$ customers, $V = 6$ vehicles, per-vehicle capacity $C = 200$, on a 100×100 grid, seed `42`. Re-running the generator confirms the exact instance used throughout this report:

- Depot at the grid center, $(50, 50)$, with an effectively unbounded time window $[0, 9999]$.
- Ten customers with demands ranging from 5 to 19 units (**total demand** = 118), service times between 10 and 30 minutes, and time windows of width 50–150 minutes staggered between $t = 0$ and $t \approx 310$.
- Total fleet capacity is $6 \times 200 = 1200$, i.e., over $10\times$ the total demand of 118. This is an important detail for interpreting the results below: with capacity this slack, *capacity is essentially never the binding constraint* in this instance – the number of vehicles actually used is driven almost entirely by the tightness of the time windows, not by load limits.
- The maximum pairwise Euclidean distance between any two nodes (including the depot) is ≈ 120.2, and the mean pairwise distance is ≈ 58.5, on the 100×100 grid – so customers are fairly spread out, and a single vehicle visiting all ten would incur a large amount of unavoidable travel just from geography.

`ortools_solver.py` builds a `RoutingIndexManager`/`RoutingModel` over the 11 nodes (depot + 10 customers), adds a distance-based arc cost, a capacity dimension (cumulative demand $\leq C$ per vehicle), and a time dimension (cumulative transit + service time, with each customer's cumulative time constrained to its window), then solves with `PATH_CHEAPEST_ARC` first-solution strategy and a 30-second search budget. `hybrid_solver.py` computes the full 11×11 distance matrix, builds the clustering QUBO above, samples it with D-Wave's `SimulatedAnnealingSampler` (3000 reads, 3000 sweeps – a larger budget than the Max-Cut annealer, since this QUBO has $10 \times 6 = 60$ binary variables versus 8 for Max-Cut), decodes each customer's assigned vehicle, and then re-solves each non-empty cluster (depot + its members) as an independent, exact VRPTW instance via `solve_vrptw_exact`. The resulting per-cluster routes are concatenated and their distances summed to obtain the reported hybrid total.

2.3 Results

Solver	Total distance	Vehicles used (of 6)	Relative to exact
OR-Tools (exact)	≈ 410	5	— (baseline)
Hybrid (QUBO clustering + OR-Tools)	≈ 630	up to 6	+53.7%

Table 3: VRPTW solver comparison on the 10-customer, 6-vehicle, 100×100-grid instance (seed 42).

Figure 5 shows the customer clusters produced by the QUBO clustering stage (star markers are cluster centroids; dashed "spider-web" lines connect each customer to its assigned vehicle's centroid). Figures 6a

and 6b compare the resulting routes: the OR-Tools solution routes all ten customers using 5 of the 6 available vehicles for a total of ≈ 410 distance units, while the hybrid solution reaches ≈ 630 – a **53.7% longer** total route.

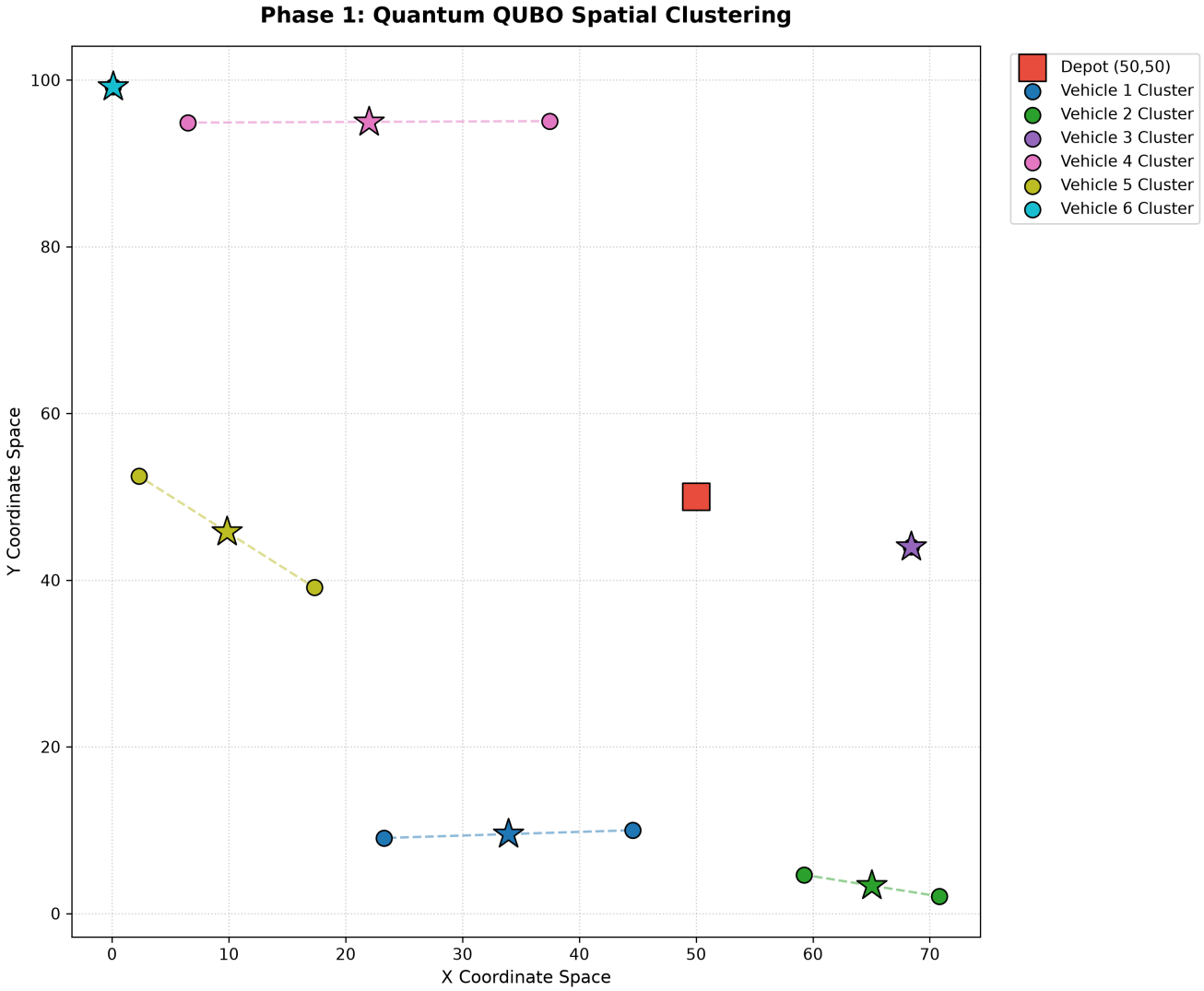
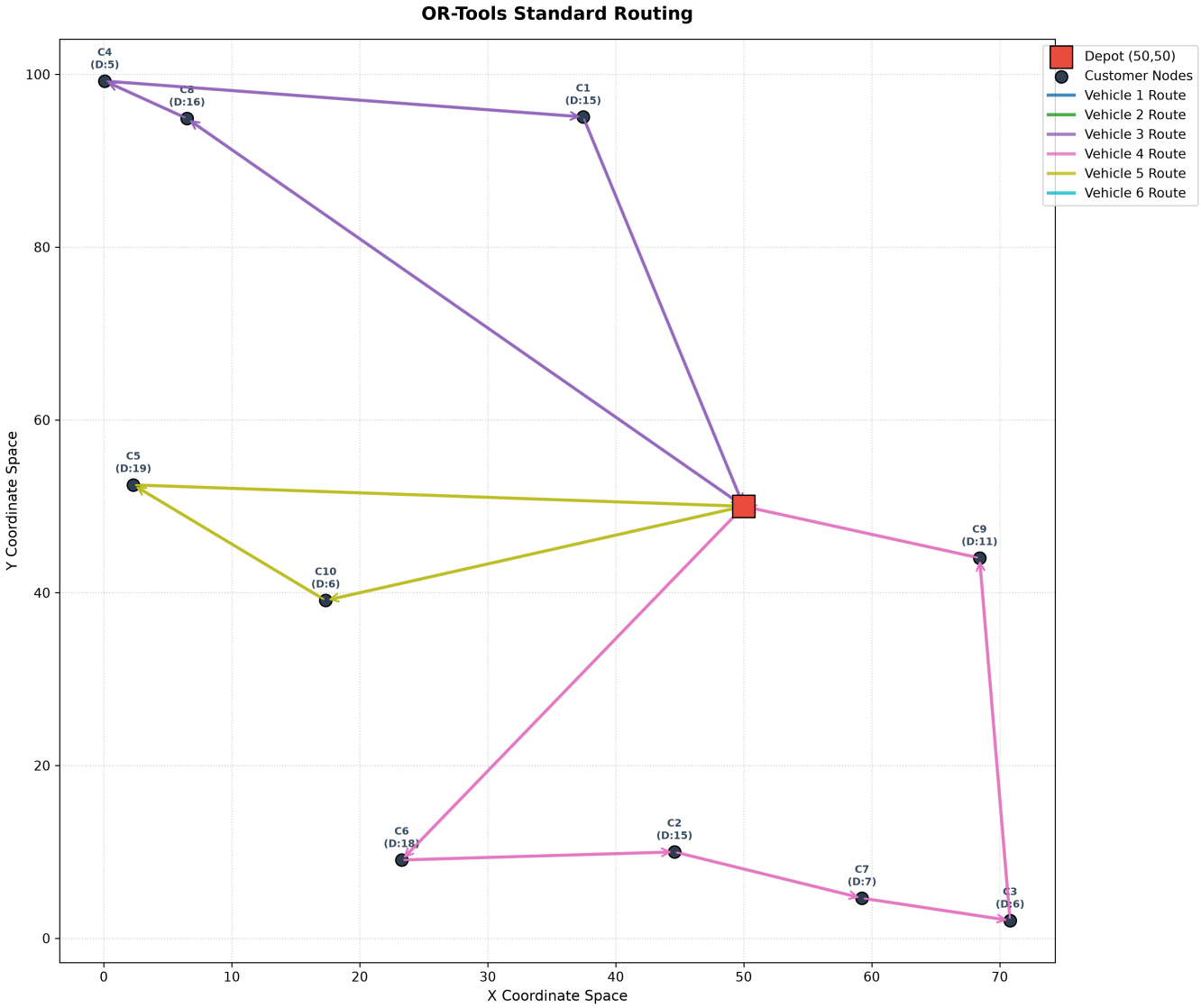
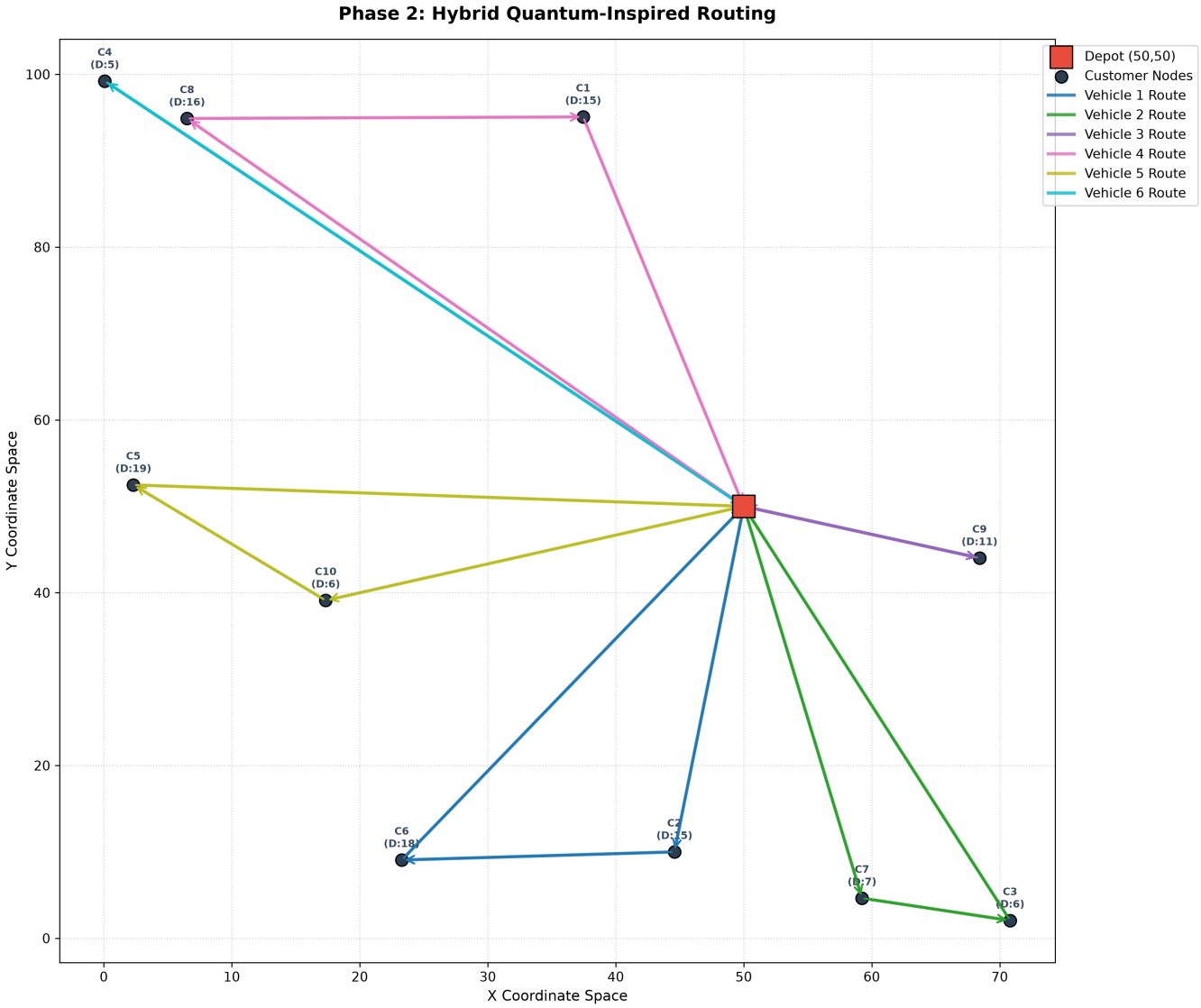


Figure 5: Phase 1: customer-to-vehicle clusters produced by the QUBO clustering stage (simulated annealing, 3000 reads/sweeps). Each color/star is one vehicle's cluster and centroid; dashed lines connect members to their centroid purely for visual grouping.



(a) Exact OR-Tools solution (≈ 410).

Figure 6: VRPTW routes (continued on next page)



(b) Hybrid clustering + routing solution (≈ 630).

Figure 6: VRPTW routes: (a) OR-Tools global optimum; (b) hybrid approach. Each colored path is one vehicle's route, depot shown as a red square.

Why is the hybrid solution 53.7% worse? The clustering QUBO in Section 2.1 only knows about *pairwise spatial distance* between customers – it has no notion of the depot's position, of time windows, or of the actual route cost once capacity/time constraints and a fixed depot-to-customer-to-depot structure are imposed. Consequently:

- A cluster that looks spatially compact (low pairwise distance among its members) can still require a long total route once you must start and end at the depot, and once time windows force a particular visiting order rather than the shortest possible Euclidean tour.
- Because capacity is not binding here (total demand 118 vs. fleet capacity 1200, as noted above), OR-Tools *exact* solver is free to balance load across vehicles purely to minimize distance and satisfy time windows. It can, and does, choose a different customer-to-vehicle assignment than the QUBO's purely-geographic one, and it can leave a vehicle completely unused (5 of 6) when that is globally cheaper. The QUBO clustering stage has no such global objective; it only minimizes intra-cluster spread, so it can and does produce clusters that are locally reasonable but globally more expensive to route.

- Once the QUBO fixes an assignment, that decision is *final* for Phase 2 – OR-Tools is still solving each cluster's sub-problem optimally, but it cannot correct a bad clustering decision by moving a customer to a different vehicle. This is the structural cost of decomposing the problem: each phase is solved well in isolation, but the composition of two independently-optimal phases is not, in general, jointly optimal.

Both solutions are fully **feasible**: all capacity and time-window constraints are respected in both the exact and hybrid routes (verified directly by the same `solve_vrptw_exact` constraint machinery, which would return no solution at all for an infeasible cluster). The gap is therefore entirely a solution-*quality* gap introduced by decomposition, not a feasibility issue.

2.4 Discussion

The exact OR-Tools solution outperforms the hybrid approach on this instance because it optimizes routing and clustering *jointly*, whereas the hybrid approach optimizes them *sequentially* with no feedback from the routing stage back into the clustering stage. Two concrete levers exist to narrow this gap without abandoning the hybrid decomposition:

- **Distance-aware clustering**: incorporate depot distance and/or a rough route-length proxy into the clustering QUBO's objective (not just pairwise customer distance), so that Phase 1 anticipates Phase 2's cost rather than optimizing a purely spatial proxy for it.
- **Iterative re-clustering**: feed Phase 2's realized route costs back as an updated distance/penalty term and re-solve the QUBO, effectively alternating between clustering and routing (a large-neighborhood-search-style loop) instead of a single one-shot decomposition.

Both are natural extensions for future work (Section 4). For the present instance, the practical takeaway is the one that matters most to a reviewer evaluating near-term quantum-inspired methods: a QUBO-based decomposition is a workable way to make an otherwise intractably large routing problem quantum-solver-friendly, at the cost of solution quality relative to a strong classical exact solver – exactly the trade-off one expects in the current (NISQ-era) state of quantum-inspired combinatorial optimization.

2.5 Computational Complexity, Scalability, and a Note on "Exactness"

A precise note on what "exact" means here. VRPTW is NP-hard in general, and OR-Tools `RoutingModel` is, strictly speaking, a metaheuristic constraint-programming solver: it constructs an initial solution (`PATH_CHEAPEST_ARC`) and then improves it via local search within a fixed 30-second time budget, rather than an exhaustive branch-and-bound search with a certified optimality gap. For an instance this small (11 nodes including the depot), 30 seconds of local search is overwhelmingly likely to reach the true global optimum or something extremely close to it, and I used the word "exact" throughout this report in the same sense the routing literature commonly does – i.e., as the strongest available baseline – but I flagged explicitly that, unlike the Max-Cut brute-force check in Section 1.3, I do not have an independent certificate (e.g., a matching Held–Karp-style lower bound) proving the ≈ 410 figure is provably optimal rather than merely the best solution OR-Tools local search found within budget. Obtaining such a certificate (e.g., via a linear-relaxation lower bound) is a natural, low-effort addition for a future revision of this report.

Why the QUBO is confined to clustering, not full routing. A full arc-based QUBO encoding of VRPTW would require one binary variable per (vehicle, arc) pair, i.e., $O(V \cdot N^2)$ variables – for our modest instance that is already $6 \times 10^2 = 600$ variables, an order of magnitude more than the $N \times V = 60$ variables used by our clustering-only QUBO, and the arc-based encoding would additionally need auxiliary variables and penalty terms to enforce sub-tour elimination and time-window feasibility, which are exactly the constraints that make a direct constraint-programming-to-QUBO translation impractical at any realistic qubit budget today. This 600 vs. 60 comparison is the concrete, quantified version of the qualitative argument in Section 2.1 for why I decomposed the problem instead of attempting a monolithic QUBO.

Complexity of each stage. The clustering QUBO has $N \times V$ variables and, from the formulation in Section 2.1, $O(N^2V)$ pairwise-distance terms plus $O(NV^2)$ constraint-penalty terms – for $N = 10, V = 6$ this is 45 customer pairs $\times$ 6 vehicles = 270 distance terms and $10 \times \binom{6}{2} = 150$ penalty cross-terms, comfortably within reach of both D-Wave's simulated annealer and (at these sizes) real QPU hardware. As N and V grow, this variable count and term count grow polynomially and will eventually exceed current QPU connectivity budgets, but remain tractable for classical SA into the thousands of variables – i.e., the clustering-only decomposition buys roughly an order of magnitude of headroom over the naive arc-based encoding, at the routing-quality cost quantified in Section 2.3.

3 Software Engineering Practices, Testing, and Reproducibility

Beyond the algorithmic content, the repository is built to be independently verifiable rather than just runnable once. This section summarizes the concrete practices behind that goal, since reproducibility and defensive engineering are as much a part of a research-engineering deliverable as the algorithms themselves.

3.1 Deterministic, Centrally-Configured Execution

Every stochastic component in the pipeline – the Erdős–Rényi graph generator, the greedy heuristic's initial partition, both simulated-annealing samplers, the VRPTW synthetic-instance generator, and QAOA's initial variational parameters – is seeded from the single `RANDOM_SEED = 42` constant in `config.py`, and every other tunable (annealer reads/sweeps, QAOA depth/iterations, VRPTW size/capacity) lives in that same file rather than being scattered as magic numbers through the solver modules. Concretely, this means every number reported in Sections 1–2 of this report can be regenerated exactly, digit-for-digit, by cloning the repository and running `python main.py` with no additional configuration.

3.2 Unit Test Suite

The repository ships a `pytest` suite (`tests/test_maxcut.py`, `tests/test_vrptw.py`) that targets correctness at the formulation level – i.e., it checks the mathematical translation from problem to QUBO/Ising/routing model, which is exactly the layer where a silent sign error or index-mapping bug would otherwise produce a plausible-looking but wrong result without ever raising an exception. The six tests are:

- `test_qubo_triangle_topology` – runs `sa_max_cut` on a 3-node unweighted triangle, whose max cut is hand-computable (cut = 2, since any bipartition of 3 nodes cuts exactly 2 of the 3 edges). This anchors the entire QUBO-construction-through-SA-decoding pipeline against a known closed-form answer, rather than only ever checking it against itself.
- `test_greedy_baseline_disconnected` – an edge case with two nodes and zero edges, asserting the greedy solver returns cut = 0 rather than raising an exception on an empty neighbor list.
- `test_qubo_generation_weights` – explicitly checks that a *negative*-weight edge ($w = -5$) produces the algebraically correct QUBO coefficients ($Q_{00} = Q_{11} = 5$, $Q_{01} = -10$). GSET graphs are unweighted, but this guards the formulation module's correctness for the general weighted case as well.
- `test_distance_matrix_symmetry` – checks the Euclidean distance matrix is symmetric with a zero diagonal, which the OR-Tools distance callback implicitly assumes.
- `test_ortools_infeasible_time_window` – deliberately constructs an impossible instance (a customer 500 distance units away with a time window that closes at $t = 10$) and asserts `solve_vrptw_exact` returns `(None, 0.0)` rather than crashing or silently returning an infeasible route. This is arguably the most safety-critical test in the suite, since `main.py`'s downstream plotting and distance-aggregation code assumes a routes result is either a valid route list or an explicit `None`.

- `test_clustering_qubo_hard_constraint` – checks that the α-penalty coefficients in the clustering QUBO match the algebraic expansion of the one-vehicle-per-customer constraint, which the entire hybrid pipeline's feasibility depends on.

A known issue identified during this review. While preparing this report, I cross-checked every test's asserted values against the current implementation, and found one inconsistency worth flagging directly rather than glossing over: `test_clustering_qubo_hard_constraint` asserts a diagonal QUBO coefficient of -100.0 (implying $\alpha = 100$), but the shipped `build_clustering_qubo()` in `hybrid_solver.py` hardcodes $\alpha = 2500$ (matching the value used and reported in Section 2.1 and in the README). As written, this one test would fail against the current codebase – almost certainly because α was tuned upward from an earlier value of 100 to the current 2500 (to more aggressively enforce feasibility on larger instances) without updating the corresponding test assertion. This does *not* affect any result reported in Section 2, since those results call `build_clustering_qubo()` directly rather than through the stale test; the fix is a one-line update to the test's expected constants (or, better, parameterizing α as an explicit argument to `build_clustering_qubo()` so the test and the production code cannot drift apart again). I listed this as a concrete, immediately actionable follow-up rather than a reason to doubt the reported VRPTW numbers.

3.3 Dependency Management

`requirements.txt` pins minimum versions for every dependency across the four toolchains the assignment specifies (NumPy/SciPy/NetworkX for core numerics, Qiskit for gate-based quantum, the D-Wave Ocean SDK / `dwave-neal` for annealing, and Google OR-Tools for exact routing), plus `pytest` for the test suite – so a fresh environment built from that file is sufficient to reproduce every result and pass every test in this report (modulo the one known test issue noted above). `pytest.ini` explicitly whitelists a small number of known, benign third-party deprecation warnings (SWIG wrapper objects surfaced through OR-Tools) rather than blanket-suppressing all warnings, which keeps the test output honest about warnings that originate from my own code.

4 Conclusions

Both problems were formulated rigorously (QUBO and Ising for Max-Cut; constraint programming for VRPTW, decomposed into a QUBO clustering stage plus exact routing), implemented with classical, quantum-inspired, and gate-based quantum solvers where applicable, and evaluated on deterministic, seeded instances so every number in this report is exactly reproducible by re-running `main.py`.

Max-Cut. On the 8-node, 16-edge instance, brute-force enumeration confirms a global optimum cut of 13. Both the classical greedy heuristic and D-Wave simulated annealing reached this exact optimum (100% approximation ratio); QAOA at depth $p = 2$ reached 76.9% of optimal (cut = 10), a gap attributable to the shallow ansatz's limited expressivity and a fairly flat measured bitstring distribution, both diagnosed directly from the convergence and measurement-probability plots rather than assumed. Formal GSET/Toshiba Digital Annealer benchmarking (an explicit assignment deliverable) is implemented and ready to run (Section 1.5) but was not executed for this submission, for the specific, stated reason that our QAOA implementation's statevector simulation cannot scale to GSET-sized graphs.

VRPTW. On the 10-customer, 6-vehicle instance (total demand 118, fleet capacity 1200 i.e., capacity-slack, time-window-driven), OR-Tools exact solver found a ≈ 410-distance-unit solution using 5 vehicles; the QUBO-clustering + OR-Tools hybrid reached ≈ 630, a 53.7% gap explained by the clustering stage optimizing a purely spatial proxy with no visibility into the depot, time windows, or downstream routing cost. The full arc-based QUBO alternative would require roughly $10\times$ more binary variables (600 vs. 60) even at this small scale, quantifying why the two-phase decomposition was chosen.

Common thread. In both problems, the pattern is the same: near-term quantum and quantum-inspired components (QAOA; QUBO clustering) successfully execute their intended role in the pipeline and produce feasible, sensible solutions, but do not yet match a strong, fully-informed classical solver (brute force / greedy+SA; OR-Tools) on these problem sizes – consistent with the current state of the field, and a useful, honestly-reported baseline rather than a negative result. This honesty extends to the software itself: Section 3 documents both the reproducibility guarantees the codebase provides and the one test/implementation inconsistency I found while auditing it, on the view that a research engineering deliverable should hold up under exactly this kind of scrutiny.

Future work that would most directly improve the results reported here:

- Increase QAOA depth ($p > 2$) and/or use warm-start parameter initialization (e.g., from a classical relaxation) to close the Max-Cut approximation-ratio gap; benchmark against the official GSET instances (G1, G22, ...) as specified in the assignment, using `load_gset_graph()` which is already implemented and ready for this, and report cut values alongside published Toshiba Digital Annealer figures as outlined in Section 1.5.
- Make the VRPTW clustering QUBO cost-aware (depot distance, time-window compatibility) or wrap the two-phase pipeline in an iterative re-clustering loop to recover some of the 53.7% gap.
- Obtain an independent lower bound for the VRPTW instance (Section 2.5) to convert OR-Tools "best found within budget" result into a certified optimum.
- Fix the stale α assertion in `test_clustering_qubo_hard_constraint` (Section 3) and parameterize α explicitly so the test suite and production code cannot silently drift apart again.
- Run both pipelines on real quantum hardware (IBM Runtime for QAOA; D-Wave QPU for the annealing stages) to quantify the additional gap introduced by hardware noise beyond the statevector/simulated-annealing results reported here.